Sorting

DS 2019/2020

----------Bubble----------

Bubble-sort

- Basic principle:
  - (i, j) with $i < j$ is an <u>inversion</u> if $a[i] > a[j]$
  - While there is an inversion $(i, i+1)$ interchange $a[i]$ with $a[i+1]$

- Algorithm:

```
Procedure bubbleSort(a, n)
begin
    last ← n − 1
    while (last > 0) do
        n1 ← last−1; last ← 0
        for i ← 0 to n1 do
            if (a[i] > a[i + 1]) then
                swap(a[i], a[i + 1])
                last ← i
end
```

Bubble sort - example

	3 **2** 1 4 7 (n1 = 2)
3 **7** 2 1 4 (n1 = 3)	**2** **3** 1 4 7
3 **7** **2** 1 4	2 **3** **1** 4 7
3 **2** **7** 1 4	2 **1** **3** 4 7
3 2 **7** **1** 4	2 1 **3** **4** 7
3 2 **1** **7** 4	2 1 **3** **4** 7
3 2 1 **7** **4**	
3 2 1 **4** **7**	**2** **1** 3 4 7 (n1 = 0)
3 2 1 4 **7**	**1** **2** 3 4 7
	1 **2** 3 4 7

Bubble sort

- Analysis
  - Worst-case performance
    $a[0] > a[1] > ... > a[n-1]$
    Time for searching: $O(n-1+n-2+...+1) = O(n^2)$
    $T_{bubbleSort}(n) = O(n^2)$
  - Best-case performance: $O(n)$

----------Insertion----------

Insertion sort

- Basic principle:
  suppose $a[0..i-1]$ sorted
  insert $a[i]$ such that $a[0..i]$ becomes sorted

- Algorithm (search sequentially the position of $a[i]$):

```
Procedure insertSort(a, n)
begin
    for i ← 1 to n − 1 do
        j ← i−1 // a[0..i − 1] sorted
        temp ← a[i] // search the place of temp
        while ((j ≥ 0) and (a[j] > temp)) do
            a[j + 1] ← a[j]
            j ← j−1
        if (a[j + 1]! = temp) then
            a[j + 1] ← temp
end
```

Insertion sort

- Example
  3 **7** 2 1
  3 7 **2** 1
  2 3 7 **1**
  1 2 3 7
- Analysis
  - searching the position i in $a[0..j-1]$ needs $O(j-1)$ steps
  - worst-case scenario $a[0] > a[1] > ... > a[n-1]$
    Search time: $O(1+2+...+n-1) = O(n^2)$
    $T_{insertSort}(n) = O(n^2)$
  - best-case scenario: $O(n)$

----------Counting----------

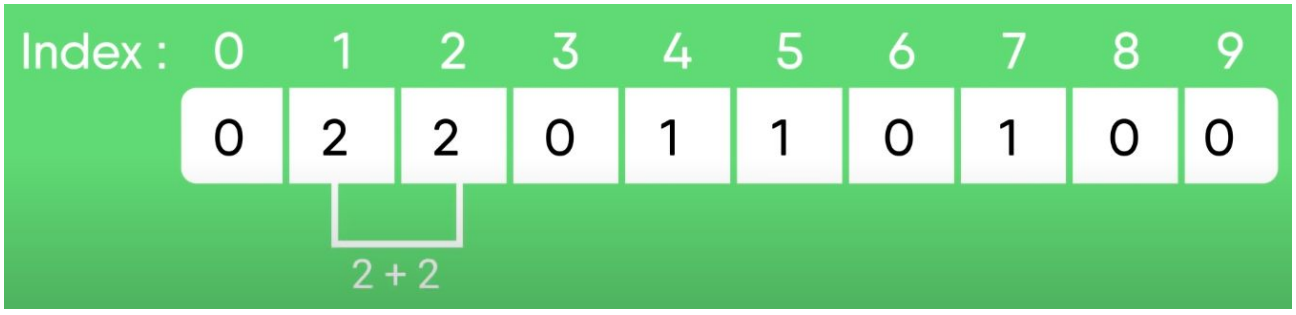
https://www.youtube.com/watch?v=7zuGmKfUt7s&ab_channel=GeeksforGeeks

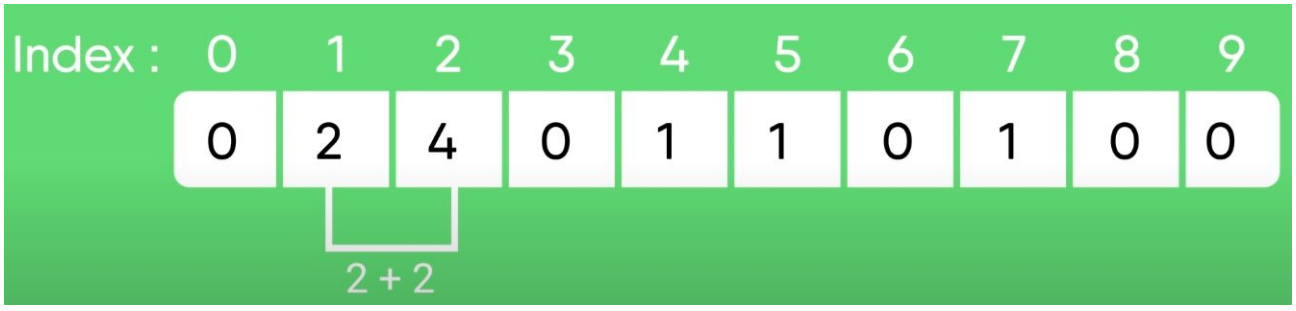
We will not dive deep into this one, the video is very good for explaining.

```
1  Procedure countingSort(a, b, n, k)
2  begin
3      for i ← 1 to k do
4          c[i] ← 0
5      for j ← 0 to n − 1 do
6          c[a[j]] ← c[a[j]] + 1
7      for i ← 2 to k do
8          c[i] ← c[i] + c[i − 1]
9      for j ← n − 1 downto 0 do
10         b[c[a[j]] − 1] ← a[j]
11         c[a[j]] ← c[a[j]] − 1
12 end
```

Idea is the following:

1. We have a vector that is not sorted
2. We initialize a count array - line 3 and 4
3. We populate the count array, line 5 and 6
4. After we create the count array we want, in each position to have “how many elements are smaller than the current element” so we will add the current position with the previous one, line 7 and 8





5. After that (line 9,10 and 11) we will begin parsing the vector that needs sorting and for every position we will see in our count vector how many elements are bigger than him, thus

we know at what position to put it (for example, if 2 has 4 elements bigger than him we know to put it at position 4 (3 if we start from 0) because at some point before him 4 number will be placed). Don't forget to decrease the count vector, line 11.

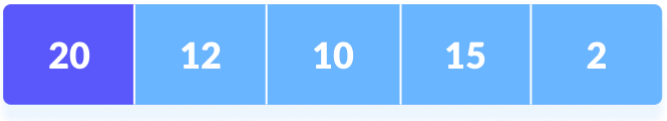
----------Selection Sort----------

- In the order $n-1, n-2, \ldots, 1, 0$, namely:
  $(\forall i) 0 \leq i < n \implies a[i] = max\{a[0], \ldots, a[i]\}$

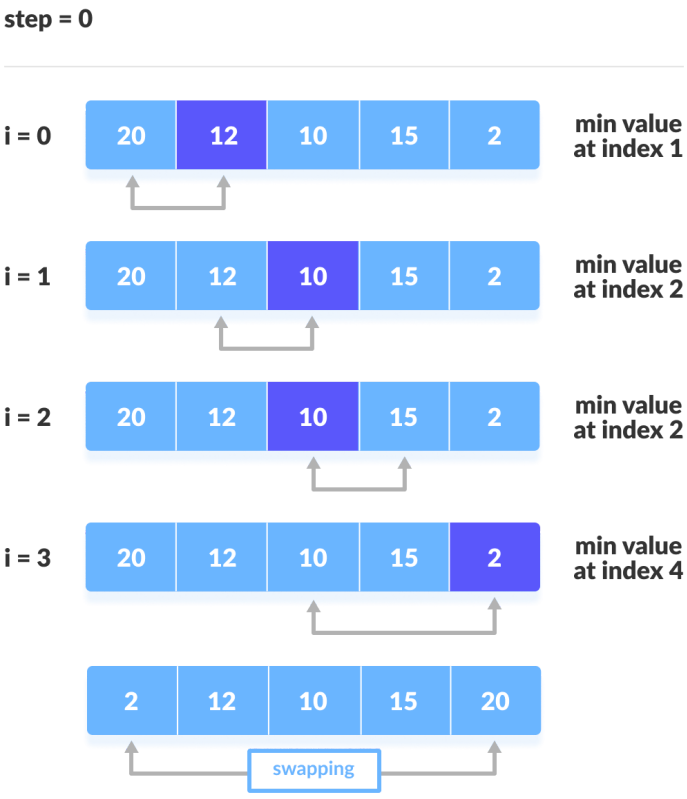
```
Procedure naivSort(a, n)
begin
    for i ← n − 1 downto 1 do
        imax ← i
        for j ← i − 1 downto 0 do
            if (a[j] > a[imax]) then
                imax ← j
        if (i! = imax) then
            swap(a[i], a[imax])
end
```

- The time complexity for all cases is $O(n^2)$

1. Set the first element as **minimum**.



2. For each iteration, compare the current position with the next, and if the next item is smaller, change the minimum.
3. After the iteration is done, we have the minimum and we put it at the end.

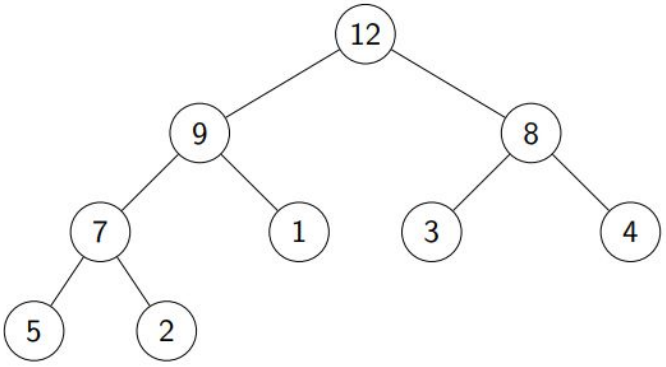


----------Heap Sort----------

Max heap = binary tree with the property that the parent has a bigger key than its children.

This means that for every subtree, the parent is the biggest element, hence the root is the biggest element in the tree.

Example of max-heap:



How can we sort using a max heap?

1. We get the array of numbers that need sorting.
2. We suppose that the array is a tree, following the property that a[k] is the parent and it's children are on a a[2k+1] and a[2k+2]
3. We will organise the tree as a max heap, hence on a[0] we will have the biggest number.
4. We will swap a[0] with the last element and recreate the max heap **without** the final, biggest element.
5. We do this again until only one "node" remains.

Example:

1. max heap

10	17	5	23	7	(n = 5)	Input array
10	**17**	5	23	7		Max heap creation using the insertTth method
10	23	5	17	7		
23	10	5	17	7		
23	17	5	10	7	(max-heap n)	Created max heap

Biggest element in the array

2. swap the first position with the last then recreate the max-heap

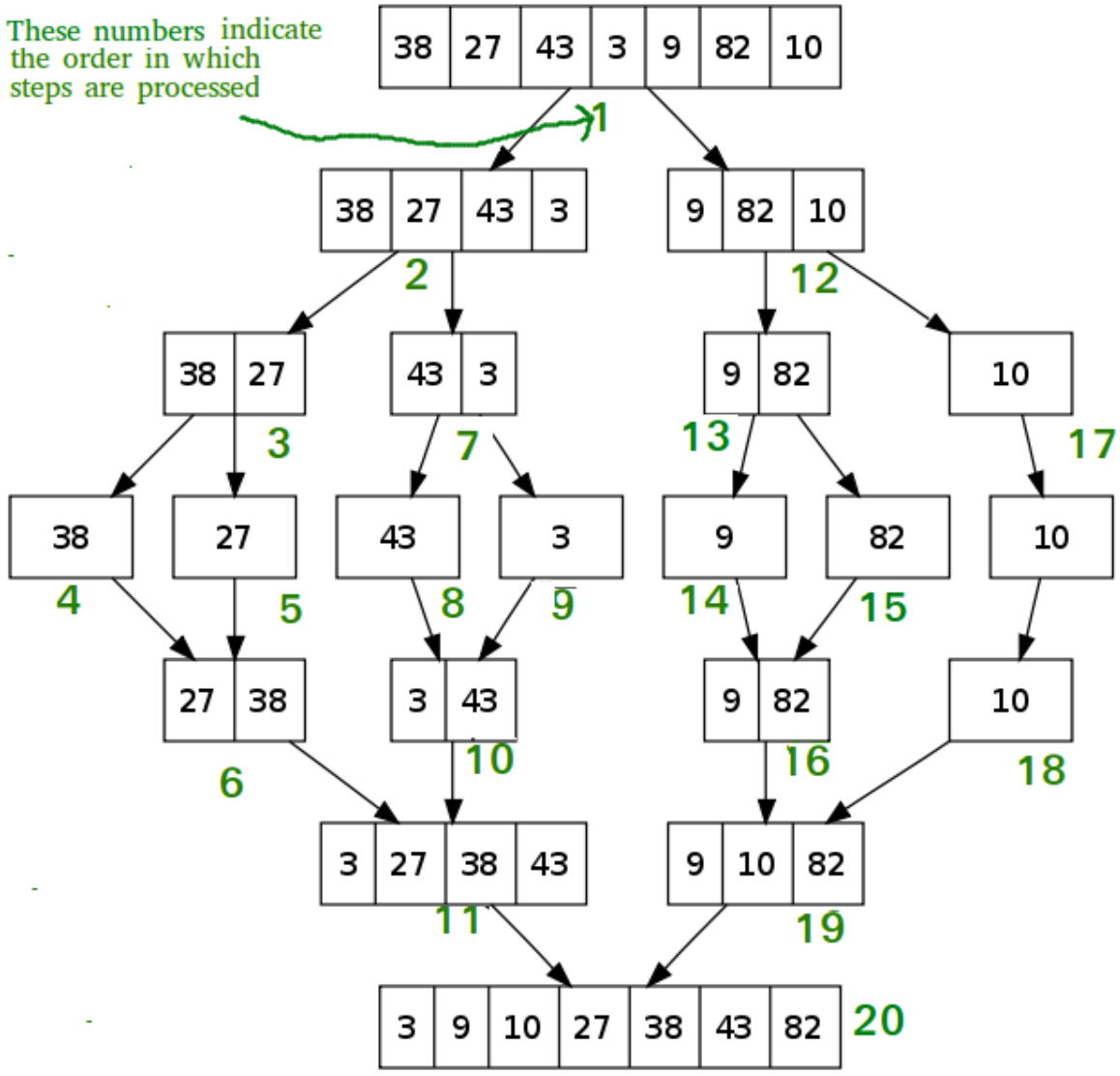
Swap

23	17	5	10	7	(max-heap n)
7	17	5	10	~~23~~	Reduce vector
17	10	5	7	~~23~~	Recreate the max heap
7	10	5	~~17~~	~~23~~	
10	7	5	~~17~~	~~23~~	(max-heap n-2)
5	7	10	~~17~~	~~23~~	
7	5	~~10~~	~~17~~	~~23~~	(max-heap n-3)
5	~~7~~	~~10~~	~~17~~	~~23~~	
5	~~7~~	~~10~~	~~17~~	~~23~~	(max-heap n-4)
5	7	10	17	23	Sorted vector

----------MergeSort----------

MergeSort(arr[], l, r) → https://www.geeksforgeeks.org/merge-sort/

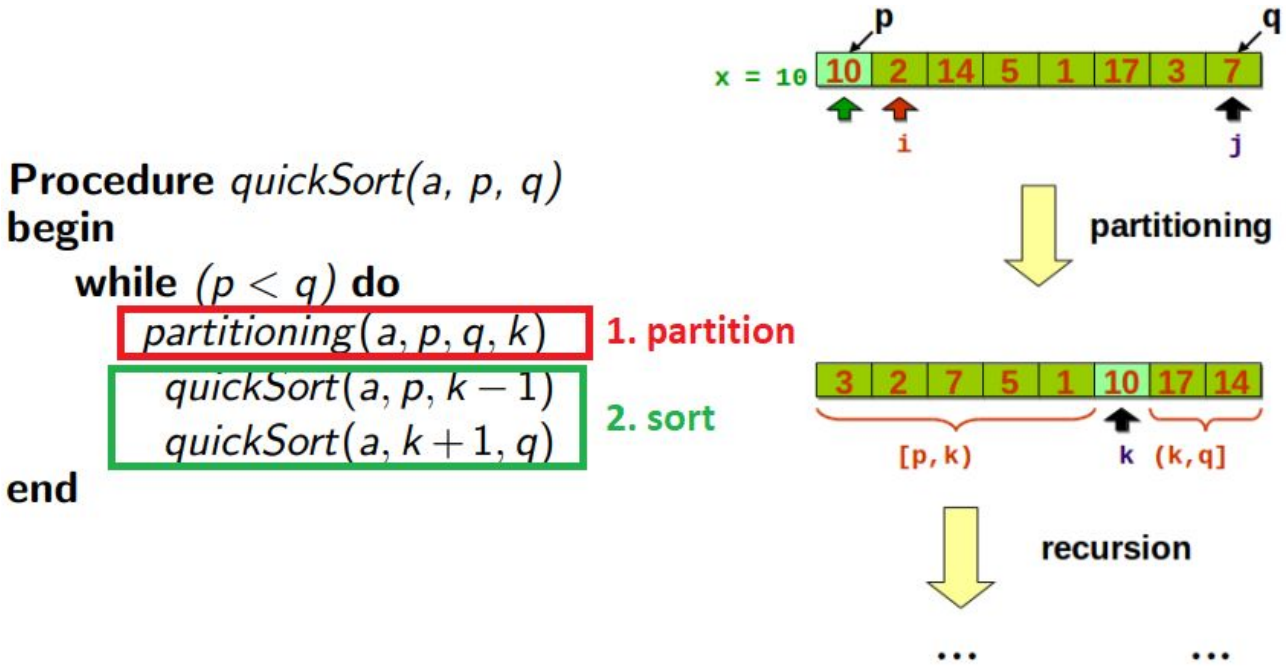
1. **Find the middle point to divide the array into two halves:**
   middle m = (l+r)/2
2. **Call mergeSort for first half:**
   Call mergeSort(arr, l, m)
3. **Call mergeSort for second half:**
   Call mergeSort(arr, m+1, r)
4. **Merge the two halves sorted in step 2 and 3:**
   Call merge(arr, l, m, r)



----------QuickSort----------

The algorithm has 2 steps:

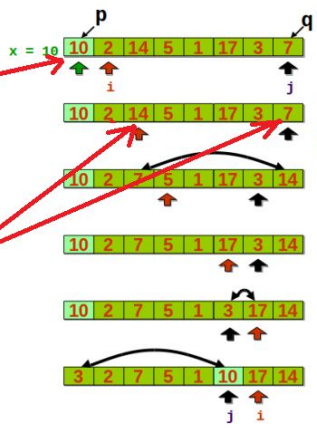
1. partitioning
2. sorting



1. In the **partitioning step**, we must move the pivot in such a way that the pivot is biggest in the [p,k) and smallest in the (k,q].
    a. Even though [p,k) <k and k < (p,k], [p,k) and (k,q] are not sorted, that is why we call step 2
2. After we partitioned the array we will call the recursive function again to partition the [p,k) array and (k,q] array.
   quicksort(a, 0,n-1)

1. In the partitioning step, we have 2 pivots and as long as the pivots are smaller and larger then k (our pivot that basically splits the vector in 2) we increase i and decrease j.
    a. If 2 elements out of place, i.e. 14 and 7, we switch them; the idea is that 14 is bigger than 10 and since it is in the “i” interval it should be smaller and 7 is smaller than k and if it is in j interval it should be bigger.
2. When we reach the end we have to put a[k] as the “border” between a[0,i] and a[j,n] thus we split the vector in 2 parts.

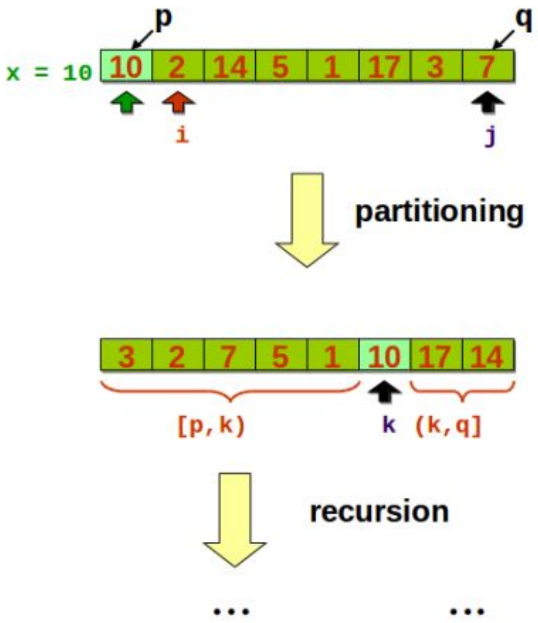
Procedure *partitioning(a, p, q, k)*
begin
$x \leftarrow a[p]$
$i \leftarrow p+1$
$j \leftarrow q$
while $(i <= j)$ **do**
if $(a[i] <= x)$ **then**
$i \leftarrow i+1$
if $(a[j] >= x)$ **then**
$j \leftarrow j-1$
if $(i < j)$ *and* $(a[i] > x)$ *and* $(x > a[j])$
then
$swap(a[i], a[j])$
$i \leftarrow i+1$
$j \leftarrow j-1$
$k \leftarrow i-1$
$a[p] \leftarrow a[k]$
$a[k] \leftarrow x$
end



7 is smaller than k and because "j" interval has only number bigger than k we must move it in the "i" interval , the opposite is available for 14

3. Knowing now that everything at left of K is smaller and at right of K bigger we can split the vector and 2 and sort 3,2,7,5,1 and 17,14

Procedure *quickSort(a, p, q)*
begin
while $(p < q)$ **do**
$partitioning(a, p, q, k)$
$quickSort(a, p, k-1)$
$quickSort(a, k+1, q)$
end



Sorting - complexity

Algorithm	Case		
	best-case	average	worst-case
bubbleSort	n	n^2	n^2
insertSort	n	n^2	n^2
naivSort	n^2	n^2	n^2
heapSort	$n \log n$	$n \log n$	$n \log n$
mergeSort	$n \log n$	$n \log n$	$n \log n$
quickSort	$n \log n$	$n \log n$	n^2
countingSort	–	$n + k$	$n + k$
bucketSort	–	n	–

Observe merge sort has “constant complexity” while quicksort is n^2 in the worst case...I think merge sort is the best between the 2.

https://www.geeksforgeeks.org/quick-sort-vs-merge-sort/